

Get Good at Operating Systems in Six Weeks

Learn OS concepts through observations and small programs. Organize the subject around virtualization, concurrency, and persistence: how programs receive CPU and memory, how work overlaps safely, and how state survives failures. A Linux VM or WSL environment is enough for most introductory experiments.

For: Developers who know how to write a program but want to understand processes, memory, scheduling, synchronization, and persistence well enough to debug and discuss real systems.

Suggested pace: 6 weeks, 6-8 hours per week | Programming foundations required

What you will be able to do

- Explain process, thread, address-space, and system-call boundaries.
- Reason about virtual memory, page faults, scheduling, and blocking I/O.
- Identify race conditions and choose an appropriate synchronization primitive.
- Distinguish a successful write from a documented durability guarantee.

Build the right mental model

Mechanism versus policy

A context switch saves and restores execution state; a scheduler chooses which runnable work executes. Page tables translate addresses; replacement policy chooses what to evict. Distinguishing mechanism from policy makes unfamiliar implementations easier to reason about.

Isolation and sharing

Processes normally have separate virtual address spaces, while threads within a process share memory and many resources. A shared address space makes communication cheap but also creates synchronization and lifetime hazards. Threads still have separate execution state and stacks.

Observe before theorizing

Use process listings, resource monitors, a debugger, and system-call tracing on your own programs. Predict whether a workload is running, blocked, or waiting for memory, then compare the prediction with evidence. Do not assume high elapsed time means high CPU use.

Study with evidence.

Read one concept, implement a tiny experiment, predict the outcome, then explain any mismatch. Keep source code and observations together. Use only local programs and disposable files; OS learning does not require probing production machines.

Week 1: Processes and execution

Study user/kernel mode, system calls, process creation, exit, and basic scheduling. Run a CPU-heavy loop and an I/O-waiting program, comparing elapsed and CPU time. Learn how file descriptors are owned and closed.

Deliverable and check: A process-state sketch and two measurements explaining why elapsed and CPU time differ.

Week 2: Virtual memory

Study address spaces, paging, translation, page faults, locality, and the difference between virtual and resident memory. Allocate memory, touch it gradually, and observe changes without exhausting the host.

Deliverable and check: A bounded experiment showing that reserving address space and physically touching pages are different events.

Weeks 3-4: Threads and coordination

Implement a shared counter and a bounded producer-consumer queue. Study mutexes, condition variables, deadlock, and safe shutdown. Explain the predicate protected by each lock and why waiting requires a loop.

Deliverable and check: A queue that does not lose work, exceed capacity, or leave a worker blocked after shutdown.

Weeks 5-6: Files and recovery

Learn page cache, filesystems, journaling, write ordering, and explicit sync operations. Study a temporary-file replacement pattern, its platform assumptions, and crash states. Review all concepts using one service's request path.

Deliverable and check: A persistence diagram and an explanation of what the application can promise after each successful call.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Diagnose a slow worker without guessing

A local worker takes five seconds to finish a request but uses very little CPU. You need to distinguish computation, waiting on a lock, sleeping, and waiting for a file or network dependency.

A worked approach

1. Measure wall-clock and CPU time over a repeatable workload. Inspect the process and its threads while the delay occurs. Low CPU does not identify the cause, but it argues against optimizing arithmetic first.
2. Collect a stack trace or system-call trace from the process you own. Look for the actual blocking boundary: a wait primitive, read, connection operation, or deliberate sleep. Preserve timestamps and avoid logging sensitive inputs.
3. Change one controlled condition, such as removing a test sleep or replacing a slow local dependency with a fast stub. Repeat the measurement and check whether the observed wait disappears without changing the result.

Expected result

You can explain the latency using a measured blocking path rather than a vague claim that the OS is slow. The distinction guides whether to optimize computation, dependency behavior, or synchronization.

Verify the result

- Is the workload identical between comparisons?
- Do stack or trace observations support the proposed bottleneck?
- Does the change preserve correctness and avoid unbounded concurrency?

Extend the exercise

Run two workers sharing a lock and deliberately hold it during slow I/O. Measure how one blocked operation delays unrelated work, then reduce the critical section and compare throughput and latency.

Make the learning stick.

Is a thread always faster than a process?

No. Threads can reduce communication and creation overhead, but introduce shared-state risks and can contend on locks. The right choice depends on isolation, workload, runtime, and failure boundaries, not a universal speed rule.

Does a page fault always mean disk I/O?

No. Some faults allocate a new page or establish an existing mapping without disk access. Others require fetching data. Explain the fault type and backing state before concluding that storage is the bottleneck.

Does write success guarantee persistence?

Not necessarily. Data may be buffered in the application, kernel, or device. A durability claim depends on the API, synchronization, filesystem, and hardware contract; a generic successful write is not enough.

Avoid these traps

- Memorizing definitions without tracing a real program.
- Using volatile as a substitute for synchronization.
- Claiming durability without identifying the persistence boundary.

Ready for the next step?

- A process and thread can be compared using resource ownership.
- A page fault and context switch can be explained separately.
- A queue has a safe wait predicate and shutdown path.
- A file update has explicit crash and durability assumptions.

Reliable next reads

- [Operating Systems: Three Easy Pieces](#)
- [Linux manual pages](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.